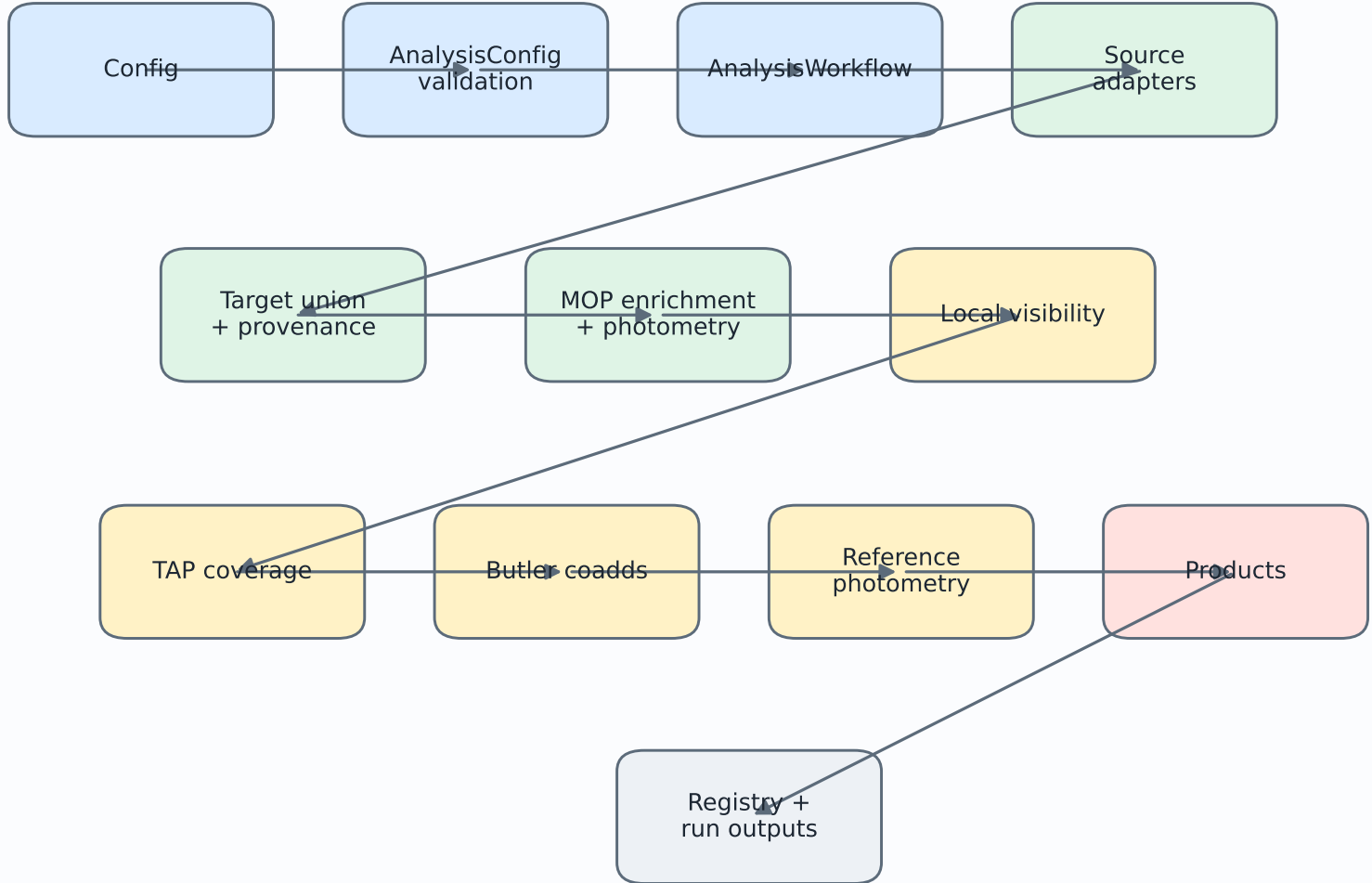


Target Selection Architecture Proposal

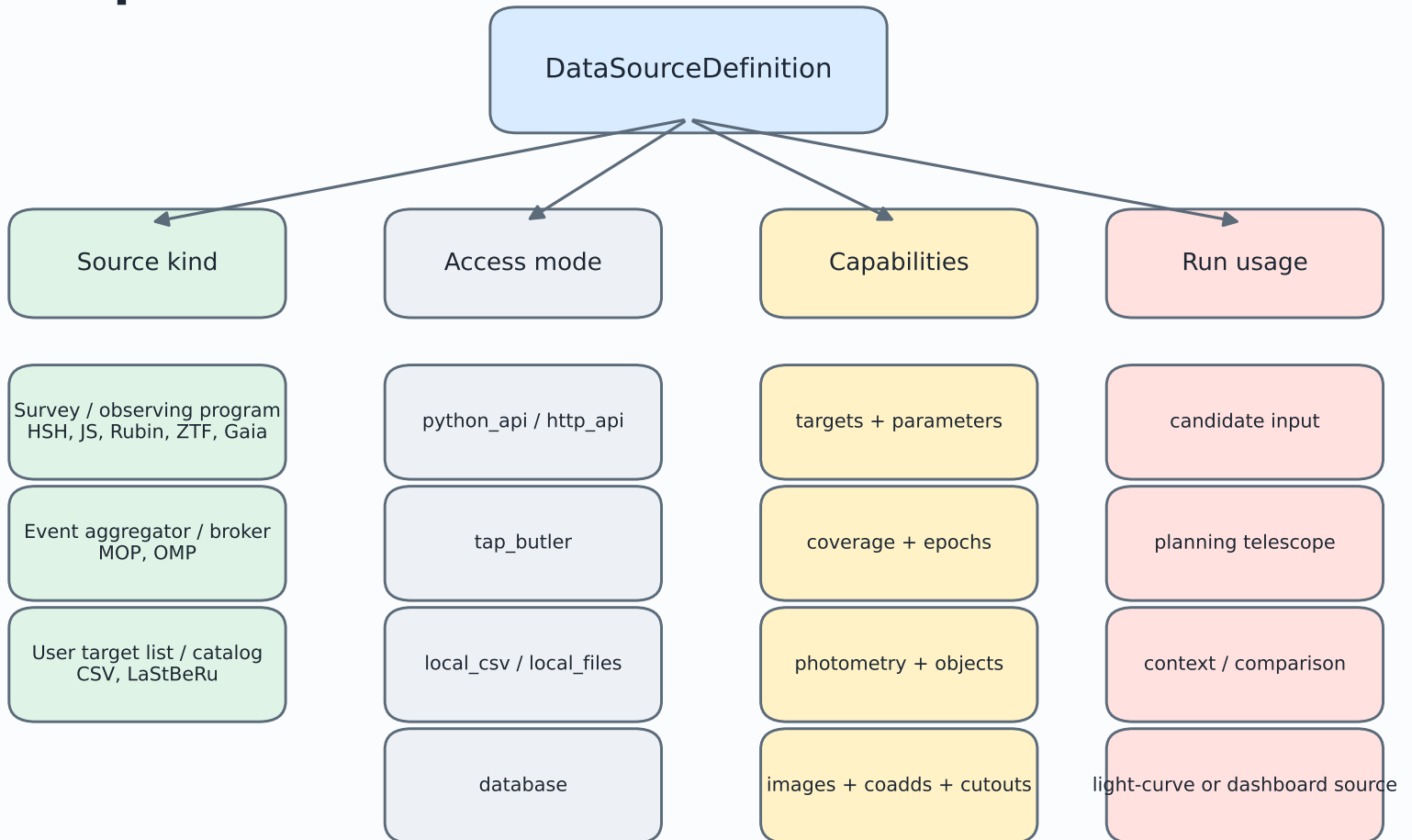
This PDF describes the current organization of target_selection and proposes an incremental architecture that is easier to extend to MOP, OMP, HSH, JS, Rubin DP1/DP2, OGLE, Gaia, ZTF, local CSVs, and future sources. The central recommendation is to define sources by kind, access mode, and capabilities, then choose their run usage explicitly.

Current Workflow



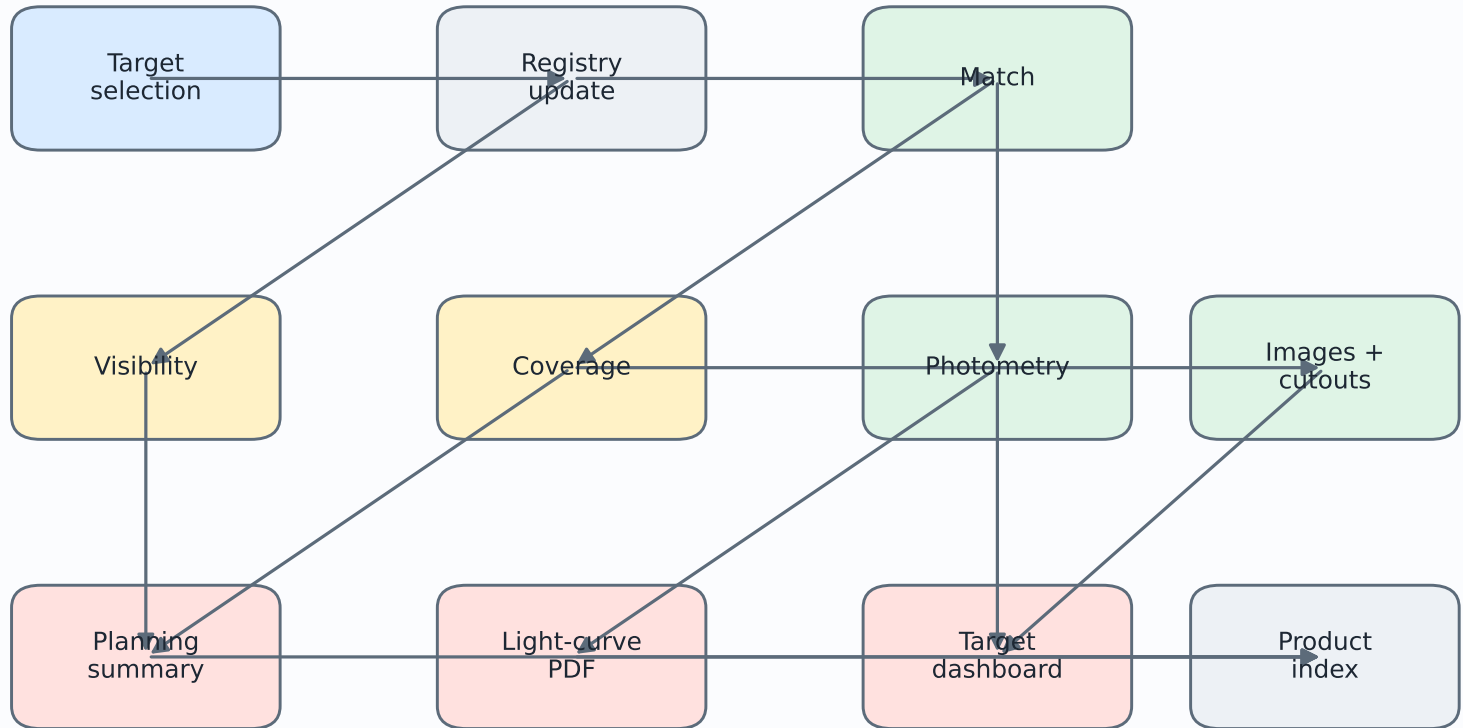
Main current issue: the high-level source/adapters model is good, but `target_selection_pipeline.py` still owns too many unrelated tasks: visibility, Rubin coverage, coadds, photometry, summaries, sky plots, and dashboards.

Proposed Source Model



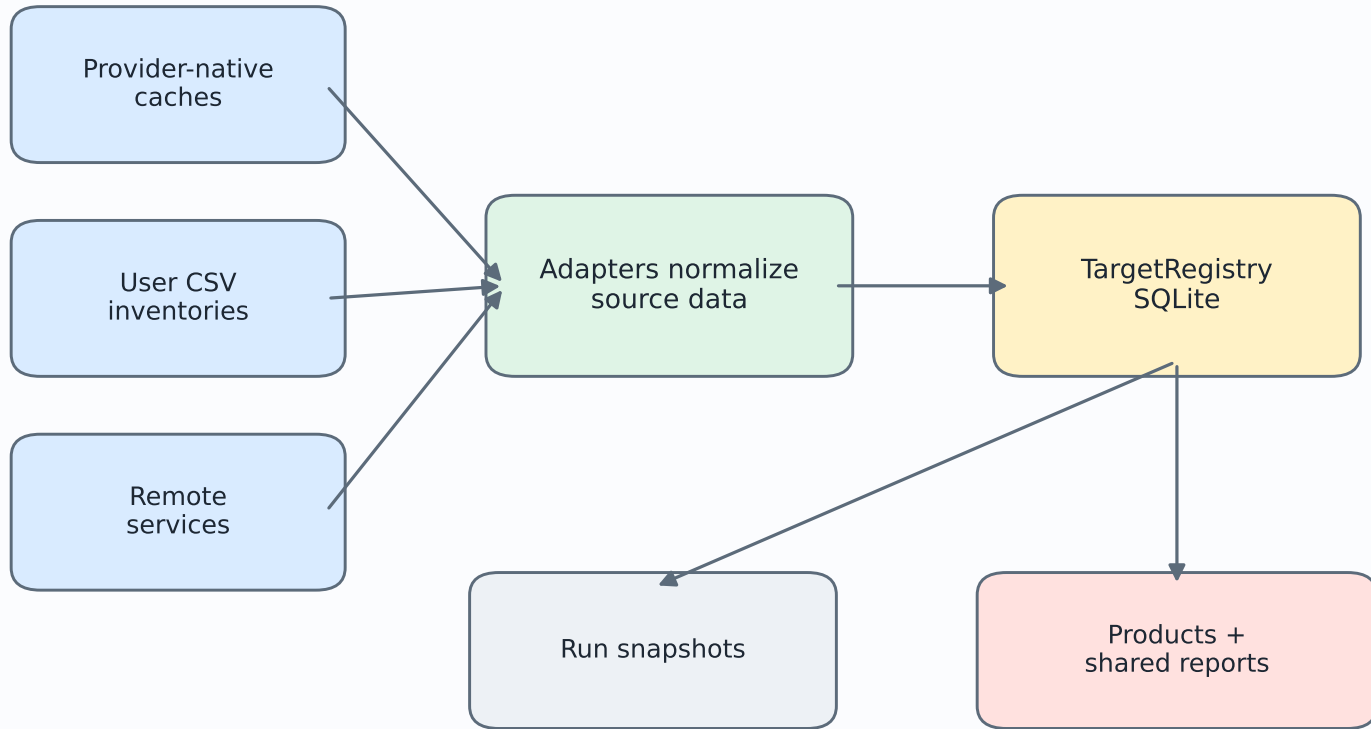
Core change: reference/follow-up are not source classes. They describe how a survey is used in one run. The stable source definition is kind + access mode + capabilities; run usage is selected separately.

Proposed Task Model



Each task should declare required inputs, optional inputs, outputs, and compatible source capabilities. This keeps dashboard-only, visibility-only, and photometry-only runs independent.

Data Ownership



Normalized persistent facts belong in the registry: identities, authoritative coordinates, source records, epochs, photometry, matches, coverage summaries, and run metadata. Large provider-native files and images can remain as files with registry metadata.

Configuration and Functional Tags

Recommended config direction:

1. Define each data source once with kind, access mode, and capabilities.
2. Select run usage separately: candidate_inputs, planning_surveys, context_surveys, photometry_sources, and image_sources.
3. Keep selection settings independent: target_names, bands, magnitude cuts, visibility dates/windows, and data-availability requirements.
4. Make tasks explicit: match, coverage, photometry, images, visibility, planning_summary, lightcurves_report, target_dashboard.
5. Keep product switches as the user-facing output layer, but allow tags such as product:target-report or photometry:dia to map to task groups.

Useful tags:

source:survey, source:aggregator, source:local-csv, targets:mop-visible, targets:observed, targets:user-list, targets:subset, match:catalog, visibility:local, coverage:survey, photometry:mop, photometry:survey, photometry:dia, photometry:coadd-forced, images:coadd, images:cutout, product:dashboard, product:lightcurves, product:planning-table, product:visibility-plots.

Refactor Recommendation

Recommended refactor path:

1. Preserve `run_analysis(config)`, the CLI, and the notebook workflow.
2. Define normalized source interfaces around kind, access mode, and capabilities. Do not make reference/follow-up separate source classes.
3. Define capability protocols: `TargetProvider`, `EventDataProvider`, `CoverageProvider`, `EpochProvider`, `PhotometryProvider`, `ObjectCatalogProvider`, `ImageProvider`, and `CutoutProvider`.
4. Split `target_selection_pipeline.py` into task modules for coverage, photometry, images, dashboards, summaries, and run assembly.
5. Promote MOP enrichment and photometry into provider capabilities, so MOP can be used for enrichment even when its daily visible target list is not selected.
6. Store match results and reference photometry in the registry with source, method, collection, counterpart ID, separation, status, and version.
7. Keep old product keywords as compatibility aliases while adding task/product groups.
8. Keep target reports in a shared `target_reports/` folder and update reports by target name.

Do not rewrite everything at once. The current architecture is usable. The next changes should isolate the monolithic Rubin/product backend first, because that is where most complexity is concentrated.